

Analysis: Dancing With the Googlers

Clever contestants will have noticed that this problem might be based on an actual television show. The author of the problem found himself watching an episode of *Dancing With the Stars* (the show and its distributor, BBC Worldwide, do not endorse and are not involved with Google Code Jam), and having forgotten whether any of the contestants had earned a score of at least 8 from any of the judges.

The judges on that show very often agree within 1 point of each other, and it really is surprising when they disagree by so much. There are a few key observations that help us solve this problem:

1. Any non-surprising score is expressed uniquely by an unordered triplet of judges' scores. 21 must be 7 7 7; 22 must be 7 7 8; 23 must be 7 8 8.
2. There only a few kinds of surprising scores. 21 could be 6 7 8; 22 could be 6 8 8; 23 could be 9 7 7. There's a repeating pattern of surprising scores that extends from 2 (0, 0, 2) to 28 (8, 10, 10).
3. The repeating pattern stops at 2 and 28 because 1 and 29 can't be surprising: we can't use (-1, 1, 1) for 1: -1 is not allowed as a score. We also can't use (9, 9, 11) for 29.

Putting all of these facts together, we can easily construct a mapping from each total score to its *best result* if it's surprising, and to its best result if it isn't surprising:

```
unsurprising(0) = 0
unsurprising(1) = 1
unsurprising(2) = 1
unsurprising(n) = unsurprising(n-3) + 1
unsurprising(n) = ceiling(n/3), for 0 <= n <= 30
```

```
surprising(2) = 2
surprising(3) = 2
surprising(4) = 2
surprising(5) = 3
surprising(n) = surprising(n-3) + 1
surprising(n) = ceiling((n-1)/3) + 1, for 2 <= n <= 28
```

Many contestants wrote an initial solution that didn't take into account that 0, 1, 29 and 30 can't be surprising, and so failed one of our sample inputs. That's what they're there for!

One way to build the tables of `unsurprising(n)` and `surprising(n)` that doesn't require so much thought could involve writing three loops to go through all possible sets of judges' scores, checking whether each combination of three was valid or surprising, and building the maps that way.

Now, for each value t_i we have one of three cases:

- `unsurprising( $t_i$ ) >= p`. This Googler can be "good" (i.e. have a maximum score of at least p) even with an unsurprising triplet.
- $2 \leq t_i \leq 28$ and `surprising( $t_i$ ) >= p > unsurprising( $t_i$ )`. This Googler can be "good" only by using a surprising triplet.
- Otherwise, the Googler can't be "good".

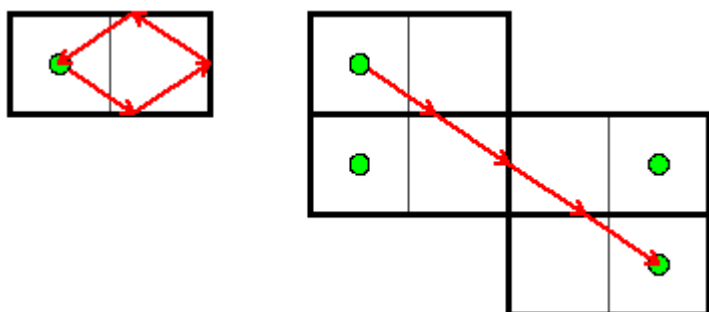
A simple greedy algorithm works: take all of the Googlers of the first type, and as many as possible (at most S) of the second type.

Analysis: Hall of Mirrors

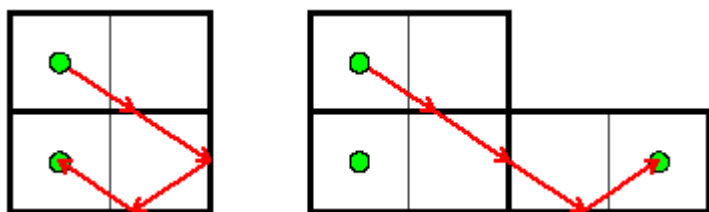
To keep things exciting, we ended the qualification round with Hall of Mirrors: a truly challenging problem. It is difficult to see how to get started, and then it is even more difficult to actually get the implementation right!

How To Understand Mirrors

The first step in solving this problem is reducing the set of all light ray angles to something that you can actually check. And to do that, it is helpful to use a famous trick for thinking about mirrors. Look at the two diagrams below. Do you see why they are really illustrating the exact same thing?



In the left picture, there is a light ray in a 2x1 room that reflects off three mirrors, and then ends back where it started. Let's focus on where the light ray reflects off the bottom mirror. Instead of drawing the reflected light ray, let's take the *entire room*, and draw it reflected about this mirror. We can then draw the rest of the light ray's path in this picture. This is illustrated in the left diagram below:



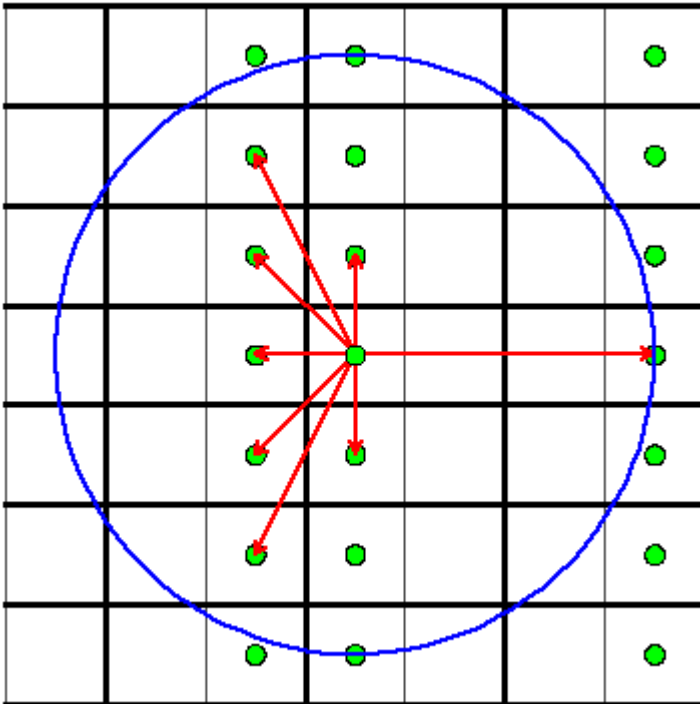
Next, once the light ray hits the right mirror, we can again draw the entire room reflected about that mirror, and then continue drawing the rest of the path there. Finally, we can do the same thing for the one remaining mirror to get the original picture from above. This kind of drawing has two really nice properties for us:

- The light ray is perfectly straight, and it never bends or turns.
- No matter how many reflections we do, and no matter where they are, the reflected position of your starting point will always be in the middle of some square.

Together, these imply the key fact you need to get started: for a light ray to even have a chance of reflecting back to you in distance at most **D**, it must be directed towards the center of a square that is distance at most **D** from you.

The Small Input

If you get this far, you can either try to solve just the small input or both the small and the large. The small input isn't that much *smaller* really, but it is *simpler* because you cannot have mirrors in the middle of rooms. This means that we can take the original room and repeatedly reflect it about each of the four walls to cover the whole plane. This is shown below:



Now let's look at your position in each of these reflected rooms. We claim that these give precisely the directions you can look in order to see a reflection of yourself. Try tracing through a few cases and you will see why.

So to solve the problem, you can iterate over all of these positions that are within distance **D** of you, and count how many unique directions they give. (Remember that the light ray stops once it returns to you, so if two reflections are in precisely the same direction, only one is visible.)

The Large Input

The large input is not any harder conceptually, but you will have to do some more implementation. The idea is to iterate over all squares within distance **D** of your starting location, and to follow light rays that are directed towards each of these squares. In particular, how many of these light rays return to the starting position after distance **D**? The challenge of following rays through a 2-dimensional grid is called "ray casting", and solving it efficiently was the key to Wolfenstein 3D and other games of that era.

This isn't too hard in theory, but the implementation can get nasty if you do not set things up carefully. One approach that helps is to first focus on rays that are moving more vertically than they are horizontally. Then, iterate over each horizontal line the ray touches. Between each of these steps, it will touch 0 or 1 vertical lines, making the processing fairly straightforward (except for corners!).

If you are stuck, try looking through the submitted solutions.

Analysis: Recycled Numbers

Many contestants got stuck in this problem because of the sample test case number 4. Let's say n is 1212, then after moving 1 or 3 digits you will get 2121, hence the pair (1212, 2121) will be counted twice if you count all possible moves. You can avoid this by breaking out of the loop once you reach the original number again, which will happen after moving 2 digits in the above example.

For the small input you can simply check for each pair (n, m) , $A \leq n < m \leq B$, whether it satisfies the conditions mentioned in the problem statement, and whether you can obtain m by moving some digits from the back of n to the front without changing their order. To check if you can obtain m from n simply try to move all possible groups of digits from n and check if what you get is m . The digit shifting can be done using string manipulation or mathematical expressions, both will run within the time limit.

But the previous solution will not run within the time limit for the large test cases. So here is another solution which should run within the time limit. For each number n , $A \leq n \leq B$, try to move all possible groups of digits from its back to its front and check if the resulting number satisfies the conditions or not. If it does satisfy the conditions then increment the result. Don't forget to avoid counting the same number twice.

Here is a sample solution:

```
int solve(int A, int B) {
    int power = 1, temp = A;
    while (temp >= 10) {
        power *= 10;
        temp /= 10;
    }
    int result = 0;
    for (int n = A; n <= B; ++n) {
        temp = n;
        while (true) {
            temp = (temp / 10) + ((temp % 10) * power);
            if (temp == n)
                break;
            if (temp > n && temp <= B)
                result++;
        }
    }
    return result;
}
```

Analysis: Speaking in Tongues

In most Google Code Jam problems, each test case is completely separate and nothing you learn from one will be useful in another. This problem was different however:

"Googlerese is based on the best possible replacement mapping, and we will never change it. It will always be the same. In every test case."

We meant it when we said this! There really is just one mapping, and the main challenge here is figuring out what it is. Fortunately, there is a lot we can learn from the sample input and output. For example, by looking at the first word in the first line, we know that "our" becomes "ejp" in Googlerese, so 'o' -> 'e', 'u' -> 'j', and 'r' -> 'p'. If you go through the entire text, you will find that there is almost enough information to reconstruct the entire mapping:

```
'a' -> 'y'
'b' -> 'n'
'c' -> 'f'
'd' -> 'i'
'e' -> 'c'
'f' -> 'w'
'g' -> 'l'
'h' -> 'b'
'i' -> 'k'
'j' -> 'u'
'k' -> 'o'
'l' -> 'm'
'm' -> 'x'
'n' -> 's'
'o' -> 'e'
'p' -> 'v'
'q' -> '???'
'r' -> 'p'
's' -> 'd'
't' -> 'r'
'u' -> 'j'
'v' -> 'g'
'w' -> 't'
'x' -> 'h'
'y' -> 'a'
'z' -> '???'
```

We just need to figure out how to translate 'q' and 'z'. But if you read the problem statement carefully, you will notice there was one more example we gave you! "a zoo" gets translated to "y qee". This means that 'z' gets mapped to 'q'.

Next we have to figure out what 'q' gets mapped to. For this part, you need to remember that every letter gets mapped to something different. And if you look carefully, there is already a letter getting mapped to everything except for 'z'. This leaves only one possibility: 'q' must get mapped to 'z'.

And now we have the full translation mapping, and all we need to do is write a program to apply it to a bunch of text. Here is a solution in Python:

```
translate_to_english = {
    ' ': ' ', 'a': 'y', 'b': 'h', 'c': 'e', 'd': 's',
    'e': 'o', 'f': 'c', 'g': 'v', 'h': 'x', 'i': 'd',
    'j': 'u', 'k': 'i', 'l': 'g', 'm': 'l', 'n': 'b',
    'o': 'k', 'p': 'r', 'q': 'z', 'r': 't', 's': 'n',
    't': 'w', 'u': 'j', 'v': 'p', 'w': 'f', 'x': 'm',
    'y': 'a', 'z': 'q'}

for tc in xrange(1, int(raw_input()) + 1):
    english = ''.join(
        [translate_to_english[ch] for ch in raw_input()])
    print 'Case #%d: %s' % (tc, english)
```